



Magnetar-Threshold: No-Reconstruct Threshold Hash-Based Signatures and the Obstruction Map for SLH-DSA

Zach Kelling

Lux Industries

2026

Magnetar-Threshold: No-Reconstruct Threshold Hash-Based Signatures and the Obstruction Map for SLH-DSA

Zach Kelling
Lux Industries, Inc.
research@lux.network

2026

Abstract

The Lux *trustless-by-default* law requires every consensus threshold signature lane to be *no-reconstruct*: no party or coordinator ever forms the full secret key, even transiently at signing time. For the lattice lanes (Pulsar / ML-DSA, Corona / Ringtail) this is achieved by exploiting the algebraic linearity of the signature response. The hash-based lane, Magnetar (threshold SLH-DSA, FIPS 205), has no such structure: the secret seed controls a forest of one-time (WOTS⁺) and few-time (FORS) secrets bound to the public key *only* through the non-homomorphic hash SHAKE. This paper is a rigorous feasibility study of *no-reconstruct* threshold SLH-DSA.

We establish the central structural fact that makes the problem tractable *in part*: FIPS 205 verification never recomputes a pseudorandom function of the seed; it only checks that the opened one-time material hashes consistently to the single published hypertree root. Leaf secrets therefore need not be pseudorandom images of a master seed; they need only be Merkle-consistent with the root. We rank four approaches (distributed tree generation, threshold leaf opening, MPC over the hash chain, and ZK-consistent partial openings) and identify the two hard walls: (i) the SLH-DSA hypertree has 2^{63} – 2^{68} leaves and cannot be materialised; (ii) deriving a leaf from the seed is a keyed hash and admits no algebraic threshold shortcut (the Cozzo–Smart obstruction).

We then build and prove the one concretely realisable, stock-FIPS-205-verifiable, genuinely *no-reconstruct* sub-protocol: *distributed FORS-leaf threshold opening*. Leaf secrets are shared directly (no master seed exists); the combiner interpolates only the message-selected one-time leaves, each exactly one leaf wide; the assembled FORS signature is byte-identical to centralised FIPS 205 forsSign and is accepted by the unmodified FIPS 205 Algorithm 17 verifier. We isolate the precise residual obstruction: the crux is not the *no-reconstruct* property of a single opening but *Byzantine-safe distributed one-time/few-time state*—the guarantee that no WOTS⁺ keypair is ever opened twice and no FORS few-time budget is exceeded across a live, adversarially scheduled committee. We prove this reduces *no-reconstruct* threshold SLH-DSA to a distributed burn-state problem and explain why, until that state is proven Byzantine-safe, the full construction must remain fail-closed and excluded from the POLARIS_MAX certificate tier. A reference implementation accompanies the paper; its signing entry returns an explicit ErrNoReconstructUnproven and a structural gate proves the opening path is incapable of reconstructing the seed.

Contents

1	Introduction	3
2	Background: the structure of SLH-DSA	4
3	Threat model, goals, and the three hard constraints	5

4	The fundamental obstruction	6
4.1	Structural freedom: verification is root-relative	6
4.2	The two walls	6
4.3	Consequence	6
5	Approach taxonomy and ranking	7
6	Distributed FORS-leaf threshold opening	8
6.1	Construction	8
6.2	Security	8
6.3	The stateful full-signature lane	9
7	One-time material: the crux	9
8	The obstruction map	11
9	Implementation and fail-closed admission	11
10	Related work	12
11	Conclusion	12

1 Introduction

Lux finalises blocks under a post-quantum certificate whose lanes provide cryptographic-family diversity: two lattice lanes (Pulsar, an ML-DSA / FIPS-204 threshold; Corona, a Ring-LWE / Ringtail threshold) and one hash-based lane, Magnetar, instantiating threshold SLH-DSA (FIPS 205, SPHINCS⁺ [1, 2]). The hash-based lane is the genuine assumption diversifier: a break of the Module-LWE/Module-SIS problems underlying both lattice lanes does not touch a construction whose only assumption is the (second-)preimage and pseudorandomness of a hash function.

The governing design rule is the *trustless-by-default law*: every consensus threshold scheme must run with *no trusted dealer* and with *no-reconstruct signing*—no party or coordinator ever forms the full secret key or seed, even transiently. A reveal-and-aggregate path in which a combiner reconstructs the key for one signing call is, at best, a non-strict accelerator; it is not admissible to the strict trustless tiers.

The two Magnetar tracks. Magnetar is split into two tracks that must not be conflated.

- **Track A — Magnetar-Quorum (production, trustless today).** Validators hold *independent* ordinary FIPS-205 keys; each signs the same finality subject; a post-quantum STARK (P3Q) proves that a weighted quorum of stock SLH-DSA signatures verified. There is no dealer, no DKG, no shared secret, and no threshold SLH-DSA at all—the difficulty is side-stepped entirely. This is the near-term hash-based lane and is out of scope here.
- **Track B — Magnetar-Threshold (research; this paper).** A genuine *t*-of-*n* threshold SLH-DSA that emits *one* stock FIPS-205 signature with no-reconstruct signing. This is an open problem. The contribution of this paper is a rigorous design, a ranked taxonomy of approaches, one concretely-buildable and proven sub-protocol, and a precise map of what is solved versus open.

Why threshold SLH-DSA is hard. In ML-DSA and BLS the signing response is a (rounded) linear function of the secret over a ring or group; Shamir/additive shares of the secret yield shares of the response, and a threshold signature is assembled by linear combination. SLH-DSA has no such structure. The secret is an n -byte seed SK.seed ; every WOTS^+ chain base and every FORS leaf secret is $\text{PRF}(\text{PK.seed}, \text{ADRS}, \text{SK.seed}) = \text{SHAKE256}(\text{PK.seed} \parallel \text{ADRS} \parallel \text{SK.seed})[:n]$, and the public key is a Merkle hash of an astronomically large forest of such secrets. There is no linear map from shares of SK.seed to the published signature.

Contributions.

1. A structural observation (Section 4) that FIPS-205 verification *never* evaluates a PRF of the seed—it only checks Merkle/chain consistency to the single published root—so leaf secrets are free to be jointly-random rather than pseudorandom images of a master seed. This is the opening that makes *threshold leaf opening* possible.
2. A ranked taxonomy of four approaches and the identification of the two hard walls—tree-size and keyed-hash leaf derivation—including the Cozzo–Smart cost bound (Section 5).
3. A concrete, no-reconstruct, stock-FIPS-205-verifiable sub-protocol: *distributed FORS-leaf threshold opening* (Section 6), with a byte-identity theorem and a leaf-width no-reconstruct theorem.
4. An exact analysis of one-time-material handling (Section 7): the reduction of no-reconstruct threshold SLH-DSA to a *Byzantine-safe distributed burn-state* for WOTS^+ one-time and FORS few-time discipline, and the argument that this is the true residual obstruction.
5. The obstruction map (Section 8) and the fail-closed admission discipline that keeps Track B out of POLARIS_MAX until proven (Section 9).

2 Background: the structure of SLH-DSA

We recall the parts of FIPS 205 [1] needed below, using the SHAKE parameter family. Fix the per-set parameters: n (hash output bytes), h (total hypertree height), d (hypertree layers), $h' = h/d$ (XMSS subtree height), a (FORS tree height, 2^a leaves per tree), k (number of FORS trees), and Winternitz $w = 16$. The recommended Lux set is SHAKE-192s ($n = 24$, $h = 63$, $d = 7$, $a = 14$, $k = 17$).

Secret key and PRF. The scheme seed expands deterministically to $(\text{SK.seed}, \text{SK.prf}, \text{PK.seed})$. Every secret leaf is derived by the keyed hash

$$\text{PRF}(\text{PK.seed}, \text{ADRS}, \text{SK.seed}) = \text{SHAKE256}(\text{PK.seed} \parallel \text{ADRS} \parallel \text{SK.seed})[:n], \quad (1)$$

where ADRS is the address of the leaf in the forest. SK.seed is the only secret input; PRF is a non-homomorphic hash of it.

WOTS^+ (one-time). A WOTS^+ keypair has $\text{len} = 2n + 3$ chains. Chain i has a secret base $\text{sk}_i = \text{PRF}(\dots)$ and public end $\text{pk}_i = \text{F}^{w-1}(\text{sk}_i)$, where F is a public hash. A signature on message digits $(b_1, \dots)$ publishes $\text{chain}(\text{sk}_i, 0, b_i) = \text{F}^{b_i}(\text{sk}_i)$ per chain. WOTS^+ is *strictly one-time*: signing two digit-vectors $b \neq b'$ reveals chains at two lengths; from the shorter one an adversary forges the other by extending the public hash—full key recovery.

FORS (few-time). FORS has k trees of 2^a leaves. Leaf secret $\text{sk}[i][j] = \text{PRF}(\dots)$; leaf node $= F(\text{sk}[i][j])$; tree i Merkle-hashes (via H) to a root; $\text{pk}_{\text{FORS}} = T_\ell(\text{root}_0 \parallel \dots \parallel \text{root}_{k-1})$. For digest md , FORS opens one leaf secret per tree at the md -selected index plus its a -node authentication path. FORS is *few-time*: signing several messages under one FORS keypair degrades security roughly as $(\text{reuse})^k$.

Hypertree and statelessness. The hypertree has d layers of XMSS trees (2^h bottom keypairs in total). A signature picks a bottom keypair $(\text{idxTree}, \text{idxLeaf})$ determined by a randomiser $R = \text{PRF}_{\text{msg}}(\text{SK.prf}, \text{PK.seed}, M')$ and digest $H_{\text{msg}}(R, \text{PK.seed}, \text{PK.root}, M')$. The huge tree makes keypair collisions negligible for a single signer over its lifetime, which is what makes SLH-DSA *stateless* and one-time-safe without explicit state.

Verification (the load-bearing fact). FIPS-205 verification (`slh_verify`, Algorithm 20) does the following and *nothing else*: parse $(R, \sigma_{\text{FORS}}, \sigma_{\text{HT}})$; recompute $md, \text{idxTree}, \text{idxLeaf}$; reconstruct pk_{FORS} from σ_{FORS} via the public hashes F, H, T_ℓ (Algorithm 17, `forsPkFromSig`); reconstruct the hypertree root from pk_{FORS} and the WOTS⁺ signatures and authentication paths; accept iff that root equals PK.root . **Verification never calls PRF and never touches SK.seed.** It checks Merkle/chain consistency to the single published root.

3 Threat model, goals, and the three hard constraints

Adversary. A static, computationally bounded, post-quantum adversary $\mathcal{A}$ corrupting up to $t - 1$ of the n committee members; full network control of message delivery and scheduling; the ability to observe all broadcast and the internal state of corrupted parties. The hash function is modelled as a (quantum) random oracle for the FIPS-205 security argument; the threshold layer adds only information-theoretic Shamir secrecy over $\mathbb{F}_{257}$.

Security goal. EUF-CMA of the produced signature under the unmodified FIPS-205 verifier, plus *key-secrecy*: the joint view of any $t - 1$ parties is independent of the global seed.

The three hard constraints (acceptance gates).

- C1. No-reconstruct.** No party or aggregator ever forms the global seed or full secret key, even transiently.
- C2. Stock verify.** The output verifies under ordinary FIPS-205 Verify, unmodified.
- C3. One-time safety.** Any revealed WOTS⁺/FORS one-time material is burned forever; no reusable secret leaks.

A subtle but decisive reading: **C1 draws the no-reconstruct line at the *global seed*, not at individual leaves.** C3 *anticipates* that one-time leaf material is revealed (“any revealed WOTS⁺/FORS one-time material. . .”). Opening a single one-time leaf secret is therefore permitted; assembling the global seed is forbidden. This distinction is what separates a viable threshold-leaf construction from a renamed seed reconstruction.

A cautionary baseline. The current production-adjacent Magnetar threshold construction (THBS-SE) and even its “dealerless” DKG *reconstruct the master seed transiently*: the combiner Lagrange-interpolates all $4n$ seed bytes into a buffer (named `derivedExpandInput / lagrangeScratch`) and runs a centralised signer. Its “strict-atom” audit gate forbids only the variable *name seed*, not the reconstruction. That is Option A (reveal-and-aggregate) and *fails C1*. Track B must do genuinely better; renaming a buffer is not a no-reconstruct proof.

4 The fundamental obstruction

4.1 Structural freedom: verification is root-relative

Proposition 1 (Root-relative verification). *Let Σ be any tuple of byte strings of the FIPS-205 signature shape. The FIPS-205 verifier accepts Σ against $(\text{PK.seed}, \text{PK.root})$ for message M iff the opened one-time values and authentication paths in Σ hash, under the public functions F, H, T_ℓ , to a hypertree root equal to PK.root . Acceptance is independent of whether the opened secrets are images of any seed under (1).*

Proof. Immediate from the verification algorithm recalled in Section 2: the only check is root equality; PRF and SK.seed never appear. $\square$

Proposition 1 is the opening. It means a committee may, at keygen, fix the forest’s secret leaves to *jointly-random* values (not PRF images) and publish a consistent root; signatures that open those leaves will still verify under the stock verifier. There is no obligation to reproduce (1). This is exactly what frees *threshold leaf opening* (approach B) from needing an MPC evaluation of PRF at signing time.

4.2 The two walls

Proposition 1 does not make the *full stateless* scheme free. Two walls remain.

Wall 1 — tree size. The hypertree has 2^h bottom keypairs (2^{63} for SHAKE-192s, 2^{64} for 256s). A jointly-random forest cannot be generated, shared, or committed in full: it is exponentially large. The randomiser R selects a keypair pseudo-uniformly over the whole tree, so a committee that has materialised only a small sub-forest cannot answer an arbitrary R without changing the scheme (and thereby breaking stock verification, which recomputes the full index range from the published R).

Wall 2 — keyed-hash leaf derivation. If instead the leaves *are* derived from a compact shared seed via (1) (so the tree need not be materialised), then producing the secret-dependent signature material requires evaluating $\text{SHAKE256}(\dots \parallel \text{SK.seed})$ on a secret-shared SK.seed . SHAKE is non-homomorphic; there is no algebraic threshold shortcut. The only generic route is multiparty computation of the Keccak permutation on shared state.

Remark 1 (Cozzo–Smart cost). *Cozzo and Smart [3] analysed the cost of thresholdising each NIST round-2 signature and concluded that hash-based schemes (then SPHINCS⁺) require MPC over the hash function, which is prohibitively expensive. A single SLH-DSA signature evaluates on the order of 10^6 – 10^7 hash compressions (dominated by recomputing WOTS⁺ public keys for authentication-path siblings); the Keccak-f permutation has ≈ 1600 AND gates per round and 24 rounds. Evaluating $\sim 10^7$ permutations in MPC is $\gtrsim 10^{11}$ AND gates per signature: feasible in principle, but minutes-to-hours of compute and gigabytes-to- terabytes of communication. This is a max-assurance ceremony, never a consensus hot path.*

4.3 Consequence

Theorem 2 (Stateless dichotomy). *A no-reconstruct t -of- n producer of a stock, stateless FIPS-205 SLH-DSA signature must, for each signature, either*

- (a) *materialise a jointly-random forest covering the R -selected keypair (infeasible by Wall 1 for $h \geq 63$), or*
- (b) *evaluate the leaf-deriving PRF and the secret-dependent authentication-path hashes under MPC on a shared seed (Wall 2; the Cozzo–Smart cost of Remark 1).*

Equivalently, route (b)—generic MPC over SHAKE—is the only path to a stock, stateless, no-reconstruct SLH-DSA signature, and it is a max-assurance ceremony, not a hot-path primitive.

Proof sketch. By Proposition 1 the leaves may be jointly random, but answering an arbitrary R requires either the full forest (route a) or on-demand derivation from a compact secret (route b). Route (a) needs 2^h shared leaves, contradicting feasibility for $h \geq 63$. Route (b) needs PRF on a shared seed; by Wall 2 this is MPC over SHAKE, whose cost is Remark 1. No third option produces a stateless signature: the only way to avoid materialising the forest *and* avoid on-demand secret derivation is to restrict the leaf set, which is statefulness (Section 6.3). $\square$

Theorem 2 is the headline negative result: *stock stateless* threshold SLH-DSA with no-reconstruct is, in essence, generic hash-MPC. The constructive value lies in (i) the FORS component, which is realisable and proven, and (ii) the *stateful* relaxation, which escapes Wall 1.

5 Approach taxonomy and ranking

Approach	What it does / wall it hits	Verdict
A Distributed tree/seed gen	Dealerless generation of a secret-shared forest + published root. The keygen half of B. Bounded by Wall 1 (only small trees).	Viable (stateful)
B Threshold leaf opening	Share leaves directly (jointly random); open only the message-selected one-time leaves; auth paths public. No master seed. Stock-verifies by Prop. 1. Bounded by Wall 1 for the full hypertree.	Best (component / stateful)
C MPC hash-chain signing	Share the seed; MPC-evaluate PRF and the hash tree on demand (route b of Thm. 2). The only stock- <i>stateless</i> path.	Viable, max-assurance only
D ZK-consistent partial openings	Parties prove in ZK (P3Q STARK) that opened material is root-consistent, removing combiner trust. Composes with A/B/C; not a standalone signer.	Orthogonal hardening

Table 1: The four approaches. B is the most viable for a buildable, stock-verifiable component and for the stateful lane; C is the only stateless path and is max-assurance only; A is B’s keygen; D hardens any of them.

Chosen design for Lux. A hybrid of B and A, scoped two ways:

- As a *proven component*: distributed FORS-leaf opening (Section 6), buildable today and stock-FIPS-205-verifiable at the FORS boundary.
- As the *full-signature lane*: a *stateful* hash-based signature (XMSS^{MT}/LMS, SP 800-208 [5]) over a tractable tree, which escapes Wall 1; this is the NIST *Haystack* direction [4] of threshold and distributed stateful HBS. Its cost is moved into a Byzantine-safe distributed state (Section 7).

Approach C is documented as the only route to stock *stateless* SLH-DSA and is reserved for a max-assurance ceremony, never the Quasar hot path. Approach D (P3Q) is the natural consistency-hardening layer because P3Q is already the canonical Lux proof verifier.

6 Distributed FORS-leaf threshold opening

This is the concrete, no-reconstruct, stock-verifiable sub-protocol. It realises the FORS bottom of an SLH-DSA signature from a t -of- n committee.

6.1 Construction

Sharing. Each FORS leaf secret is an n -byte value. We share it byte-wise over $\mathbb{F}_{257}$ (the smallest prime > 255 ; every byte is a distinct field element). Party p at evaluation point $x_p \in [1, 257)$ holds, for each shared leaf, the vector $(f_b(x_p))_{b < n}$ where f_b is a degree- $(t-1)$ polynomial with constant term the b -th leaf byte and *independent* random higher coefficients per leaf (coefficient reuse across leaves leaks differences and is explicitly avoided).

Setup (keygen). For a FORS keypair address K and digest md , the setup provisions the k message-selected leaves: it shares each selected leaf secret and publishes the public material—the k authentication paths and $\mathbf{pk}_{\text{FORS}}$. The public auth-path nodes and $\mathbf{pk}_{\text{FORS}}$ are hashes of the secret leaves; computing them from shares without exposing unopened leaves requires either an MPC over $\mathbb{F}$ at keygen or a setup-TCB dealer (“dealer for setup only”). The reference uses the dealer pattern and *erases* all leaf material before returning; the dealerless+MPC keygen is the documented keygen cost. The SIGN-side wire shape is identical under either keygen.

Partial open. Party p releases its $\mathbb{F}_{257}$ share vectors for the k selected leaves, bound by a commitment to (K, md, p) so the shares cannot be replayed for a different keypair or message.

Combine (public). Anyone collecting $\geq t$ valid partial opens interpolates, *per leaf*, the n -byte leaf secret over $\mathbb{F}_{257}$ —guarded so each interpolation is exactly one leaf wide—then writes the leaf secrets and the public authentication paths into the FORS signature buffer. Each opened leaf address is burned in the one-time ledger before its secret is released.

Algorithm 1 OpenForsThreshold (public combiner, no-reconstruct)

Require: public material $(K, md, \{\text{authpath}_j\}_{j < k}, \mathbf{pk}_{\text{FORS}}, \mathbf{PK.secd}, t)$; partial opens $\{P_p\}$; burn ledger L

Ensure: FORS signature σ_{FORS} , byte-identical to centralised `forsSign`

- 1: validate each P_p against its commit on (K, md, p) ; collect $\geq t$ share-points per FORS tree j
 - 2: **for** $j = 0$ to $k - 1$ **do**
 - 3: **assert** interpolation width = n ▷ C1 guard: one leaf, never the seed
 - 4: $L.\text{Burn}(\text{addr}(K, j), md)$ ▷ C3: abort on reuse (slashable)
 - 5: $\text{sk}_j \leftarrow \text{LagrangeAtZero}_{\mathbb{F}_{257}}(\text{any } t \text{ share-points})$ ▷ one leaf wide
 - 6: write sk_j and authpath_j into σ_{FORS} at tree j
 - 7: **end for**
 - 8: **return** σ_{FORS}
-

6.2 Security

Theorem 3 (Byte-identity). *If the shared leaves equal the leaves of a genuine FORS keypair K and the published authentication paths equal that keypair’s, then for any t honest partial opens,*

OpenForsThreshold *outputs the byte-identical FORS signature that centralised FIPS-205 forsSign produces on the same (K, md) , and forsPkFromSig of the output equals pk_{FORS} .*

Proof. $\mathbb{F}_{257}$ Lagrange interpolation of t shares of a degree- $(t-1)$ polynomial returns its constant term, i.e. the exact leaf byte. Hence each interpolated sk_j equals the genuine leaf secret. The combiner writes $(sk_j, \text{authpath}_j)$ into the FORS signature at the same offsets forsSign uses, with identical public auth-path bytes; the outputs are byte-equal. forsPkFromSig is a deterministic public function of those bytes, so it reproduces pk_{FORS} . $\square$

Corollary 4 (Stock-FIPS-205 acceptance). *The assembled FORS signature is accepted by the unmodified FIPS-205 FORS public-key reconstruction (Algorithm 17), which is byte-equal to the cloudflare/circl reference. A full signature embedding this FORS component (with a matching hypertree part) verifies under stock SLH-DSA Verify.*

Theorem 5 (No-reconstruct, leaf-width). *OpenForsThreshold interpolates secret material only at width n (one leaf). It never forms a buffer of the $\geq 4n$ -byte seed/secret-key width, and it never invokes the leaf-deriving PRF, the seed expansion, or a centralised signer. The joint view of any $t-1$ parties is independent of every unopened leaf and of the (nonexistent) master seed.*

Proof. The combiner’s only secret-bearing operation is the per-leaf $\mathbb{F}_{257}$ interpolation at line 5 of Algorithm 1, of fixed width n , asserted at line 3 (any wider width is refused before the secret is touched). The construction has *no master seed*: leaves are shared directly, so there is no seed object to reconstruct—C1 holds vacuously for the seed and by the width-guard for any secret. Key-secrecy of each unopened leaf is Shamir information-theoretic secrecy over $\mathbb{F}_{257}$ for a degree- $(t-1)$ polynomial given $\leq t-1$ evaluations; the opened leaves are exactly the FIPS-205 one-time material a signature reveals anyway, no more. $\square$

Remark 2 (Mechanised structural gate). *The no-reconstruct property of Theorem 5 is enforced mechanically in the reference implementation by a static gate that parses the combiner and asserts its call graph excludes every seed-reconstruction / secret-side primitive (the leaf-deriving PRF, the seed expansion, the centralised signer, the share-dealing routines) and includes the leaf-width interpolation and the C1/C3 guards (Section 9).*

6.3 The stateful full-signature lane

For a complete signature, Wall 1 forbids materialising the SLH-DSA hypertree. The escape is to use a *stateful* hash-based signature over a tractable tree ($\text{XMSS}^{MT}/\text{LMS}$, $h \leq 20-25$): the committee distributively generates a secret-shared tree, publishes all public WOTS⁺ keys and internal nodes (so authentication paths are public lookups), and at sign time advances to the next unused leaf and threshold-opens only that leaf’s WOTS⁺ chain material. This is exactly threshold leaf opening (approach B) over a materialisable tree, and it is genuinely no-reconstruct at signing (only one one-time leaf is opened). Its public key is a stock SP 800-208 public key and its signatures verify under the stock LMS/ XMSS verifier. The price is statefulness, which becomes the distributed burn-state of Section 7; this is the NIST Haystack lane [4]. It is a different FIPS standard than stateless SLH-DSA and is offered as the *max-assurance hash lane*, not as FIPS-205.

7 One-time material: the crux

The deepest obstruction is not the no-reconstruct property of a single opening—the FORS component achieves it—but *global* one-time/few-time discipline across a live, possibly-Byzantine committee.

WOTS⁺ strict one-time. If the committee ever opens chain material for one WOTS⁺ address under two distinct message digests, the shorter chains let an adversary forge the longer ones: full WOTS⁺ key recovery, hence a hypertree forgery. The committee must guarantee that for every (layer, tree, leaf) WOTS⁺ address, chain material is opened for *at most one* message digest, forever.

FORS few-time. Opening leaves of one FORS keypair across several messages degrades few-time security; the budget set by (k, a) must not be exceeded.

Stateless selection becomes distributed state. In stateless SLH-DSA the keypair selection $(idxTree, idxLeaf)$ is $R = \text{PRF}_{\text{msg}}(\text{SK.prf}, \text{PK.seed}, M')$, and the 2^h tree makes reuse negligible for a single signer. In a threshold setting SK.prf is itself shared, so R must be produced by the committee, and the committee must *detect and refuse* any second opening of an already-burned one-time address. This is the “stateful subprotocol under a stateless-looking certificate boundary” of NIST Haystack [4].

Definition 1 (Burn-state). *A burn-state is a map from one-time addresses to the message digests they were consumed for, with: (idempotence) re-opening the same (addr, digest) is benign; (conflict-detection) opening an addr for a different digest is refused and yields the two-digest tuple as slashable evidence; (few-time budget) FORS addresses admit a bounded reuse count.*

Theorem 6 (Reduction to a Byzantine-safe burn-state). *Assume (i) a no-reconstruct threshold opening of every one-time leaf on the R -selected path (the FORS component of Section 6 plus its WOTS⁺ analogue over a materialisable tree), and (ii) a burn-state that is safe under $t - 1$ Byzantine corruptions and adversarial scheduling: no honest party ever co-signs an opening that would re-open a strictly-one-time WOTS⁺ address or exceed a FORS few-time budget. Then the committee produces stock-verifiable signatures satisfying C1, C2, C3.*

Proof sketch. C2 (stock verify) and C1 (no-reconstruct) follow from Theorems 3–5 applied to each opened leaf along the R -path and from Proposition 1 for the assembled root. C3 (one-time safety) is exactly assumption (ii): the burn-state refuses every reuse before the one-time secret is released, so no reusable secret leaks. The signature thus carries only the single-use material a stock signature reveals, and verifies to PK.root. $\square$

Remark 3 (The residual obstruction, stated exactly). *Theorem 6 isolates the open problem: a Byzantine-safe distributed burn-state. A local, single-process ledger (the reference implementation) trivially satisfies the burn-state definition—it refuses reuse and emits slashable conflicts—but does not survive crashes, restarts, network partitions, or Byzantine equivocation, under which two honest sub-committees could be driven to open the same WOTS⁺ address for different messages. Lifting the burn-state to consensus-anchored burn proofs (each opening gated on a finalised no-reuse certificate) is the missing piece. It is a state-machine-replication problem layered under the cryptography, and it is why the full Track-B construction must remain fail-closed: the crypto is no-reconstruct and stock-verifiable, but the one-time discipline that hash-based signatures demand is not yet proven Byzantine-safe. **One-time-material handling, not no-reconstruct opening, is the true wall for the full scheme.***

8 The obstruction map

Sub-problem	Status	Where
No-reconstruct opening of a single one-time leaf	Solved & proven (byte-identical, stock-verifiable)	Thm. 3, 5
FORS bottom of a signature, no-reconstruct	Solved & proven (component)	Section 6
Direct leaf sharing avoids any master seed	Solved (C1 vacu- ous for the seed)	Prop. 1
Stock-stateless SLH-DSA, no-reconstruct	Open / max- assurance only : route (b) generic MPC over SHAKE	Thm. 2, Rem. 1
Full signature over a materialisable tree (stateful)	Buildable (XMSS ^{MT} /LMS, SP 800-208)	Section 6.3
Dealerless keygen of the public forest from shares	Open / costly : MPC over $\mathbb{F}$ at keygen (else setup- TCB dealer)	Section 6.1
Byzantine-safe distributed one-time/few-time state	Open (the crux): consensus- anchored burn proofs	Thm. 6, the residual remark

Table 2: What is solved versus open for no-reconstruct threshold SLH-DSA.

Summary in one line. *No-reconstruct opening is solved; stock stateless SLH-DSA reduces to hash-MPC (max-assurance only); the buildable full lane is stateful HBS; and the binding obstruction for any live deployment is a Byzantine-safe distributed burn-state for one-time material.*

9 Implementation and fail-closed admission

The reference implementation accompanies this paper in the Magnetar package ([ref/go/pkg/magnetar](#)). It comprises:

- `threshold_noreconstruct.go` — the Track-B interface and the *single* admission gate. The signing entry returns `ErrNoReconstructUnproven`; `NoReconstructProven()` is hard-wired `false`; `AdmitMagnetarThresholdToPolarisMax(TrackB)` refuses. Track B can therefore never be silently admitted to `POLARIS_MAX`. It also hosts the `BurnLedger` (the burn-state of Section 7) and the constraint-1 leaf-width guard.
- `fors_threshold_open.go` — the distributed FORS-leaf opening of Section 6: setup, partial open, public combiner, and stock-FIPS-205 verification (Algorithm 17).
- Tests proving: byte-identity to centralised `forsSign` and stock acceptance across SHAKE-192s and 256s with strict t -of- n subsets (Theorem 3); one-time burn refusal with slashable evidence (Section 7); commit-mismatch rejection; the fail-closed signer and admission gate; the leaf-width guard; and a *structural AST gate* (Theorem 5, mechanised) that the combiner never calls a seed-reconstruction primitive—contrasted against the existing reveal-and-aggregate combiner, which does.

Policy. The refined Quasar policy table places Track B only in the research tier `MAGNETAR_EXPERIMENTAL`; `POLARIS_MAX` uses Track A (Magnetar-Quorum) for the hash lane. `NoReconstructProven()` stays `false` until a no-reconstruct producer for the *complete* signature and a Byzantine-safe burn-state are proven. Flipping it is a reviewed act gated on Table 2’s open rows.

10 Related work

Threshold hash-based signatures are nascent. NIST’s *Haystack* effort on threshold and distributed *stateful* HBS [4] is the closest line; its distributed-state techniques are directly relevant and motivate our *stateful* lane and burn-state. Cozzo and Smart [3] bounded the cost of thresholding NIST signatures and flagged the hash-MPC barrier we formalise in Theorem 2. The underlying primitives are SPHINCS⁺/SLH-DSA [2, 1], XMSS/LMS [6, 5], Shamir secret sharing [7], verifiable secret sharing and dealerless DKG [8, 9, 10], and generic MPC for the hash-evaluation route [11, 12]. The ZK-consistency hardening (approach D) uses transparent post-quantum STARKs [13]. In contrast to the lattice threshold lanes, where the response is linear and thresholdisation is algebraic, the hash lane has no such structure—which is precisely the gap this paper charts.

11 Conclusion

No-reconstruct threshold SLH-DSA is not a single problem but a stack. The no-reconstruct *opening* of one-time leaves is solved and proven, and we ship a stock-FIPS-205-verifiable FORS-leaf component to demonstrate it. Stock *stateless* SLH-DSA reduces to generic MPC over SHAKE and is max-assurance only. The buildable full lane is *stateful* hash-based signing over a materialisable tree. And the binding obstruction for any live, Byzantine deployment is a *distributed burn-state* that enforces WOTS⁺ one-time and FORS few-time discipline—one-time-material handling, not the threshold opening, is the wall. Until that state is proven Byzantine-safe and a complete-signature producer exists, Magnetar-Threshold stays fail-closed and excluded from `POLARIS_MAX`; the trustless hash lane in production is Magnetar-Quorum (Track A).

References

- [1] National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)*. 2024.
- [2] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, P. Schwabe. *The SPHINCS⁺ Signature Framework*. ACM CCS, 2019.
- [3] D. Cozzo, N. P. Smart. *Sharing the LUOV: Threshold Post-Quantum Signatures*. IMA Int. Conf. Cryptography and Coding, 2019.
- [4] National Institute of Standards and Technology. *Threshold and Distributed Stateful Hash-Based Signatures* (“Haystack”), NIST Threshold Cryptography (MPTC) program documents. 2023–2025.
- [5] D. Cooper, D. Apon, Q. Dang, M. Davidson, M. Dworkin, C. Miller. *NIST SP 800-208: Recommendation for Stateful Hash-Based Signature Schemes*. 2020.
- [6] A. Hülsing, D. Butin, S. Gazdag, J. Rijneveld, A. Mohaisen. *RFC 8391: XMSS—eXtended Merkle Signature Scheme*. IRTF, 2018.
- [7] A. Shamir. *How to Share a Secret*. Communications of the ACM, 22(11), 1979.

- [8] P. Feldman. *A Practical Scheme for Non-Interactive Verifiable Secret Sharing*. FOCS, 1987.
- [9] T. P. Pedersen. *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*. CRYPTO, 1991.
- [10] R. Gennaro, S. Jarecki, H. Krawczyk, T. Rabin. *Secure Distributed Key Generation for Discrete-Log Based Cryptosystems*. EUROCRYPT, 1999.
- [11] O. Goldreich, S. Micali, A. Wigderson. *How to Play Any Mental Game*. STOC, 1987.
- [12] I. Damgård, V. Pastro, N. Smart, S. Zakarias. *Multiparty Computation from Somewhat Homomorphic Encryption (SPDZ)*. CRYPTO, 2012.
- [13] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity (STARK)*. ePrint 2018/046.